



Pilas y Colas en Pythonz

Estructuras de datos fundamentales que todo programador debe dominar



¿Qué son las Pilas y Colas?

Pilas (Stacks)

Estructuras de datos que siguen el principio **LIFO** (Last In, First Out). Como una pila de platos: el último que se coloca es el primero que se retira.

Estas estructuras son fundamentales en la informática por su simplicidad y eficiencia en operaciones específicas. Su comprensión es esencial para el desarrollo de algoritmos más complejos.

Colas (Queues)

Estructuras que siguen el principio **FIFO** (First In, First Out). Como una fila de personas: el primero en llegar es el primero en ser atendido.



Ejemplos en la Vida Real



Deshacer/Rehacer

Los editores de texto utilizan pilas para almacenar las acciones realizadas. Cada vez que presionas Ctrl+Z, se ejecuta un "pop" de la pila de acciones.



Cola de Impresión

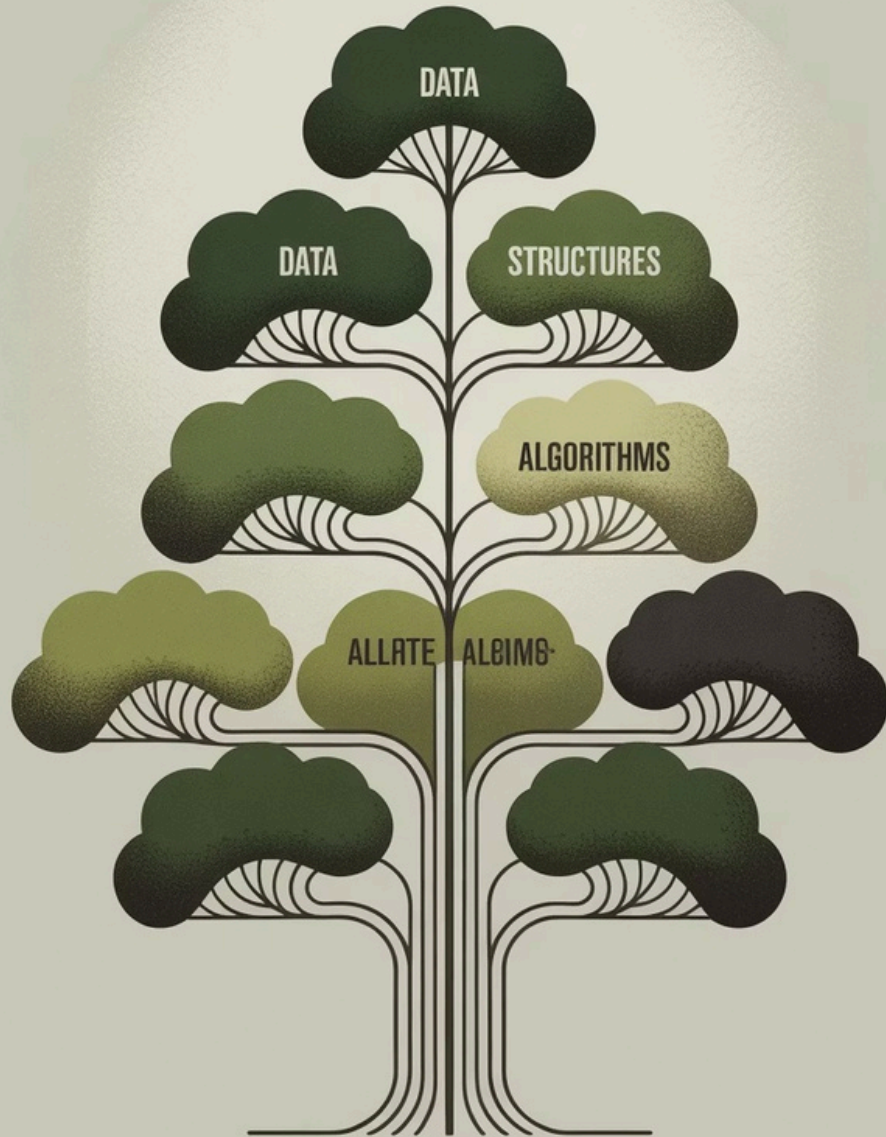
Los documentos se imprimen en el orden que fueron enviados a la impresora. El primer documento enviado es el primero en imprimirse.



Buffers de Datos

Los sistemas operativos usan colas para gestionar procesos en espera y pilas para almacenar llamadas a funciones en la memoria.

Clasificación de Estructuras de Datos



Estructuras Lineales

- Pilas (Stacks)
- Colas (Queues)
- Listas enlazadas
- Arrays

Estructuras No Lineales

- Árboles binarios
- Grafos
- Tablas hash
- Heaps

Las pilas y colas pertenecen a las estructuras lineales, donde los elementos se organizan secuencialmente. Esta característica las hace ideales para operaciones que requieren un orden específico de acceso a los datos.



Aplicaciones Avanzadas

Algoritmos de Backtracking

Las pilas permiten explorar soluciones de manera sistemática, retrocediendo cuando se encuentra un callejón sin salida. Esencial en problemas como el laberinto o las 8 reinas.

Planificación de Procesos

Los sistemas operativos utilizan colas para gestionar procesos en espera, implementando algoritmos como Round-Robin y First-Come-First-Served.

Recorrido de Grafos

BFS (Breadth-First Search) usa colas para explorar nivel por nivel, mientras que DFS (Depth-First Search) utiliza pilas para profundizar primero.

Repaso: Listas en Python

Antes de implementar pilas y colas, repasemos las operaciones fundamentales de las listas de Python, que servirán como base para nuestras estructuras personalizadas.

■ `append()`

Añade un elemento al final de la lista. Operación $O(1)$ amortizada.

■ `pop()`

Elimina y devuelve el último elemento. También $O(1)$.

■ `insert()` y `remove()`

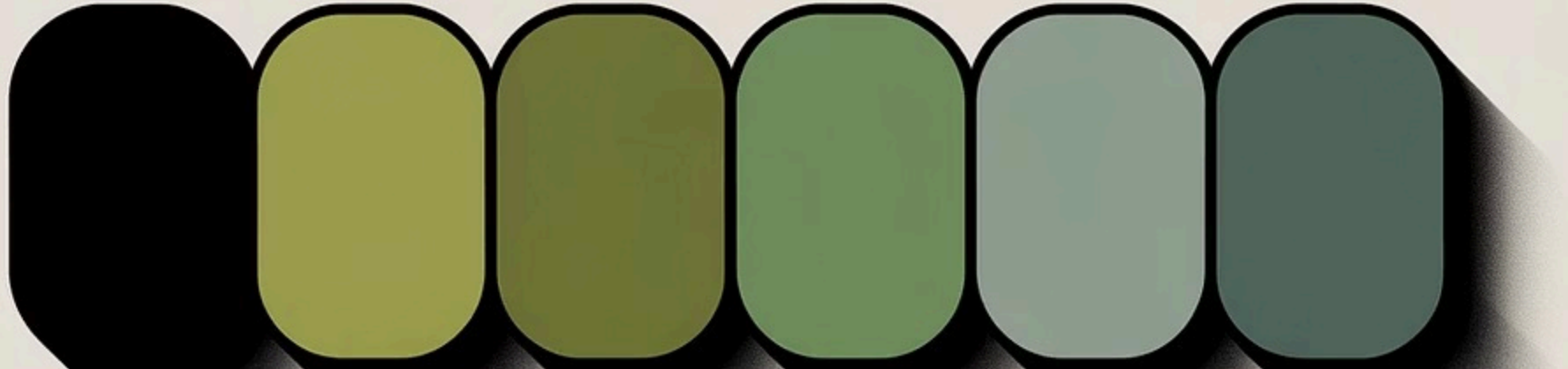
Operaciones $O(n)$ que requieren desplazar elementos.



Complejidad Temporal de Listas

Operación	Complejidad	Explicación
append()	$O(1)^*$	Amortizada
pop()	$O(1)$	Desde el final
pop(0)	$O(n)$	Desplaza elementos
insert(0, x)	$O(n)$	Desplaza todos
Acceso [i]	$O(1)$	Acceso directo

Las operaciones al final de la lista son eficientes, pero las operaciones al inicio requieren desplazar todos los elementos, resultando en una complejidad lineal.



El Módulo `collections.deque`

Python proporciona `collections.deque` (double-endedqueue) que ofrece operaciones $O(1)$ en ambos extremos, siendo mucho más eficiente que las listas para implementar colas.

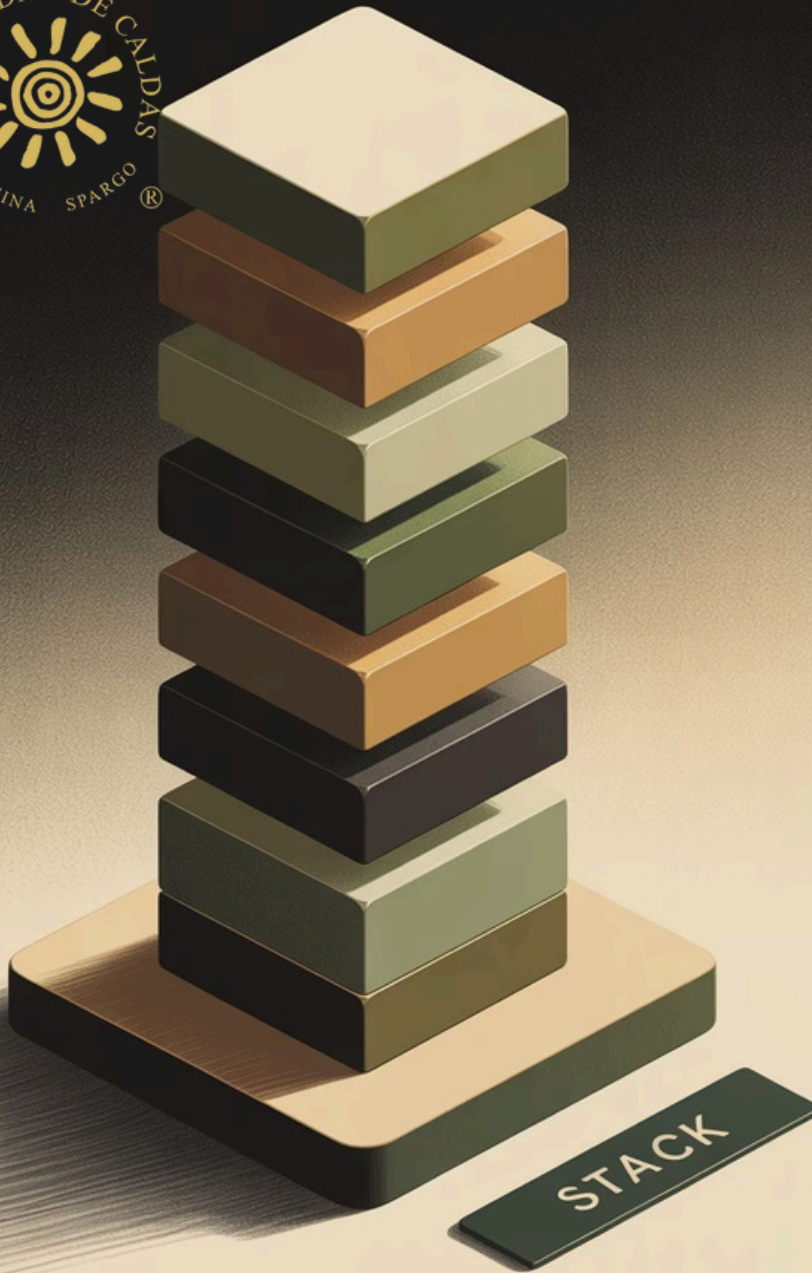
```
from collections import deque

# Creación
mi_deque = deque([1, 2, 3])

# Operaciones eficientes en ambos extremos
mi_deque.appendleft(0) #  $O(1)$ 

mi_deque.append(4)     #  $O(1)$ 
mi_deque.popleft()     #  $O(1)$ 
mi_deque.pop()         #  $O(1)$ 
```

Esta estructura es la base recomendada para implementar tanto pilas como colas en Python de manera eficiente.



Pilas (Stacks) - Definición

Principio LIFO

Last In, FirstOut - El último elemento añadido es el primero en salir. Como una pila de platos en una cafetería.

Operaciones Básicas

- **push()**: Añadir elemento
- **pop()**: Quitar elemento
- **peek()**: Ver el tope
- **is_empty()**: Verificar si está vacía

Las pilas son estructuras restrictivas: solo se puede acceder al elemento del tope. Esta limitación es en realidad su fortaleza, proporcionando un comportamiento predecible y eficiente.

Implementación de Pila desde Cero

```
class Stack:
    def __init__(self):
        """Inicializa una pila vacía"""
        self.items = []

    def push(self, item):
        """Añade un elemento al tope de la pila"""
        self.items.append(item)

    def pop(self):
        """Elimina y devuelve el elemento del tope"""
        if self.is_empty():
            return None
        return self.items.pop()

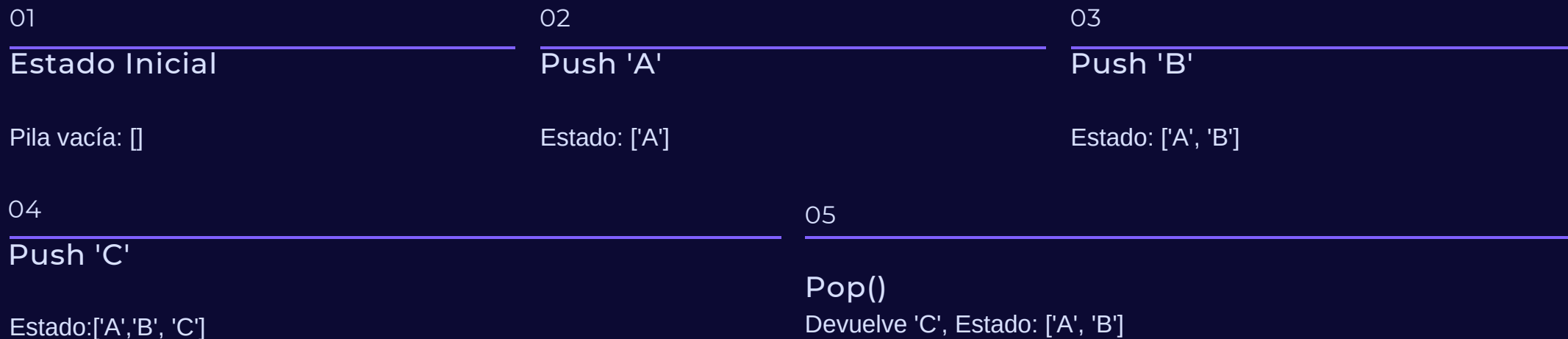
    def peek(self):
        """Devuelve el elemento del tope sin eliminarlo"""
        if self.is_empty():
            return None
        return self.items[-1]

    def is_empty(self):
        """Verifica si la pila está vacía"""
        return len(self.items) == 0

    def size(self):
        """Devuelve el número de elementos"""
        return len(self.items)
```



Diagrama de Operaciones de Pila



Ejemplo Práctico: Conversión Decimal a Binario

```
def decimal_a_binario(numero):  
    """Convierte un número decimal a binario usando una pila"""  
    if numero == 0:  
        return "0"  
  
    pila = Stack()  
    # Dividir por 2 y apilar los residuos  
  
    while numero > 0:  
        residuo = numero % 2  
        pila.push(str(residuo))  
        numero = numero // 2  
  
    # Desapilar para formar el número binario  
    binario = ""  
    while not pila.is_empty():  
        binario += pila.pop()  
  
    return binario  
  
# Ejemplo de uso  
print(decimal_a_binario(23)) # Output: "10111"
```

La pila nos permite invertir el orden de los dígitos binarios, ya que necesitamos el último residuo calculado como primer dígito del resultado.

Ejemplo Avanzado: Balance de Paréntesis

```
def verificar_parentesis(expresion):  
    """Verifica si los paréntesis están balanceados"""  
    pila = Stack() pares = {'(': ')', '[': ']', '{': '}'}  
    for caracter in expresion:  
  
        if caracter in pares: # Es un paréntesis de apertura  
            pila.push(caracter)  
        elif caracter in pares.values(): # Es un paréntesis de cierre  
            if pila.is_empty():  
                return False # Cierre sin apertura  
  
            apertura = pila.pop()  
            if pares[apertura] != caracter:  
                return False # No coinciden los tipos  
  
    return pila.is_empty() # Verdadero si no quedan aperturas  
  
# Ejemplos  
print(verificar_parentesis("()[]{}")) # True  
print(verificar_parentesis("([{}])")) # True  
print(verificar_parentesis("([])")) # False
```

Este algoritmo es fundamental en compiladores y editores de código para validar la sintaxis correcta.



Análisis de Eficiencia de Pilas

Operación	Complejidad	Justificación
push()	$O(1)$	Inserción al final
pop()	$O(1)$	Eliminación del final
peek()	$O(1)$	Acceso directo
is_empty()	$O(1)$	Verificación de longitud

Todas las operaciones fundamentales de una pila tienen complejidad constante, lo que las convierte en estructuras muy eficientes para sus casos de uso específicos.

Colas (Queues) - Definición

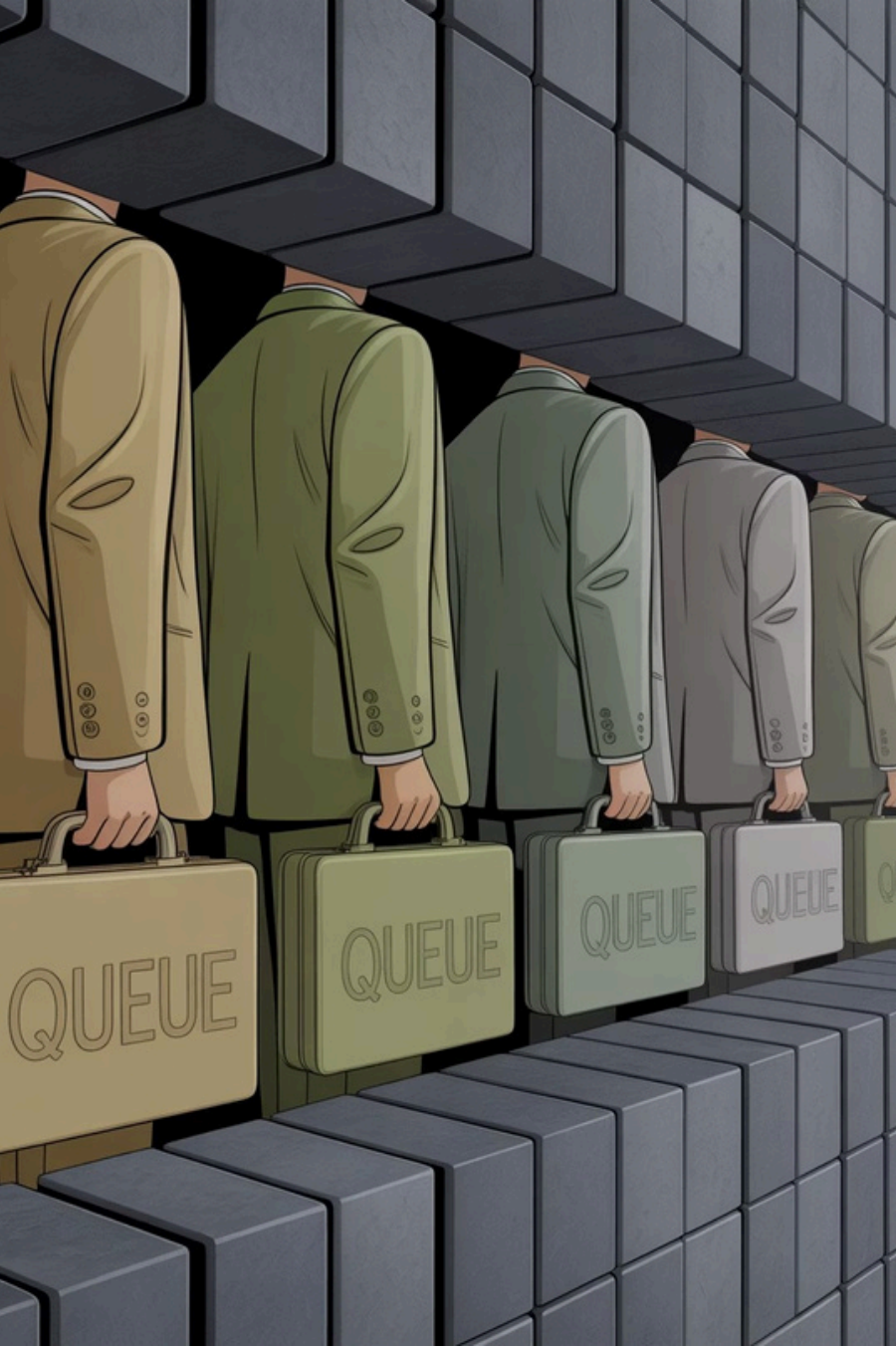
Principio FIFO

First In, First Out - El primer elemento añadido es el primero en salir. Como una fila en el supermercado.

Operaciones Principales

- **enqueue()**: Añadir al final
- **dequeue()**: Quitar del frente
- **front()**: Ver el primer elemento
- **rear()**: Ver el último elemento

Las colas mantienen el orden de llegada, siendo fundamentales en sistemas que requieren procesamiento justo y ordenado de elementos.



Implementación Ineficiente con Listas

```
class QueueIneficiente:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item) # O(1) - eficiente

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0) # O(n) - ¡INEFICIENTE!
        return None
```

- ❏ **¡Cuidado!** La operación `pop(0)` es $O(n)$ porque debe desplazar todos los elementos restantes hacia la izquierda. Para una cola con 1000 elementos, cada `dequeue` requiere 999 operaciones de movimiento.

Esta implementación puede causar problemas de rendimiento graves en aplicaciones con muchas operaciones de cola.

Implementación Eficiente con deque



```
from collections import deque

class Queue:
    def __init__(self):
        """Inicializa una cola vacía usando deque"""
        self.items = deque()

    def enqueue(self, item):
        """Añade un elemento al final de la cola - O(1)"""
        self.items.append(item)

    def dequeue(self):
        """Elimina y devuelve el primer elemento - O(1)"""
        if self.is_empty():
            return None
        return self.items.popleft()

    def front(self):
        """Devuelve el primer elemento sin eliminarlo"""
        if self.is_empty():
            return None
        return self.items[0]

    def rear(self):
        """Devuelve el último elemento sin eliminarlo"""
        if self.is_empty():
            return None
        return self.items[-1]

    def is_empty(self):
        """Verifica si la cola está vacía"""
        return len(self.items) == 0
```

Ejemplo Práctico: Simulación de Fila de Banco

```
import random
import time

def simular_banco():
    """Simula el sistema de turnos de un banco"""
    cola_clientes = Queue()
    tiempo_total = 0
    clientes_atendidos = 0

    # Simular Llegada de clientes
    nombres = ["Ana", "Luis", "María", "Carlos", "Elena"]
    for nombre in nombres:
        print(f"{nombre} llega al banco")
        cola_clientes.enqueue(nombre)
        time.sleep(0.5) # Simular tiempo entre llegadas

    # Procesar clientes
    while not cola_clientes.is_empty():
        cliente_actual = cola_clientes.dequeue()
        tiempo_atencion = random.randint(2, 5)

        print(f"Atendiendo a {cliente_actual} por {tiempo_atencion} minutos")
        tiempo_total += tiempo_atencion
        clientes_atendidos += 1
        time.sleep(1) # Simular tiempo de atención

    print(f"Banco cerrado. Atendidos: {clientes_atendidos}, Tiempo total: {tiempo_total} min")

# simular_banco()
```



Caso Avanzado: Round-Robin CPU Scheduling

```
class Proceso:
    def __init__(self, nombre, tiempo_cpu):
        self.nombre = nombre
        self.tiempo_restante = tiempo_cpu
        self.tiempo_original = tiempo_cpu

    def round_robin(procesos, quantum=3):
        """Implementa algoritmo Round-Robin para planificación de CPU"""
        cola_listos = Queue()
        tiempo_total = 0

        # Añadir todos los procesos a la cola
        for proceso in procesos:
            cola_listos.enqueue(proceso)

        print(f"Iniciando Round-Robin con quantum = {quantum}")

        while not cola_listos.is_empty():
            proceso_actual = cola_listos.dequeue()

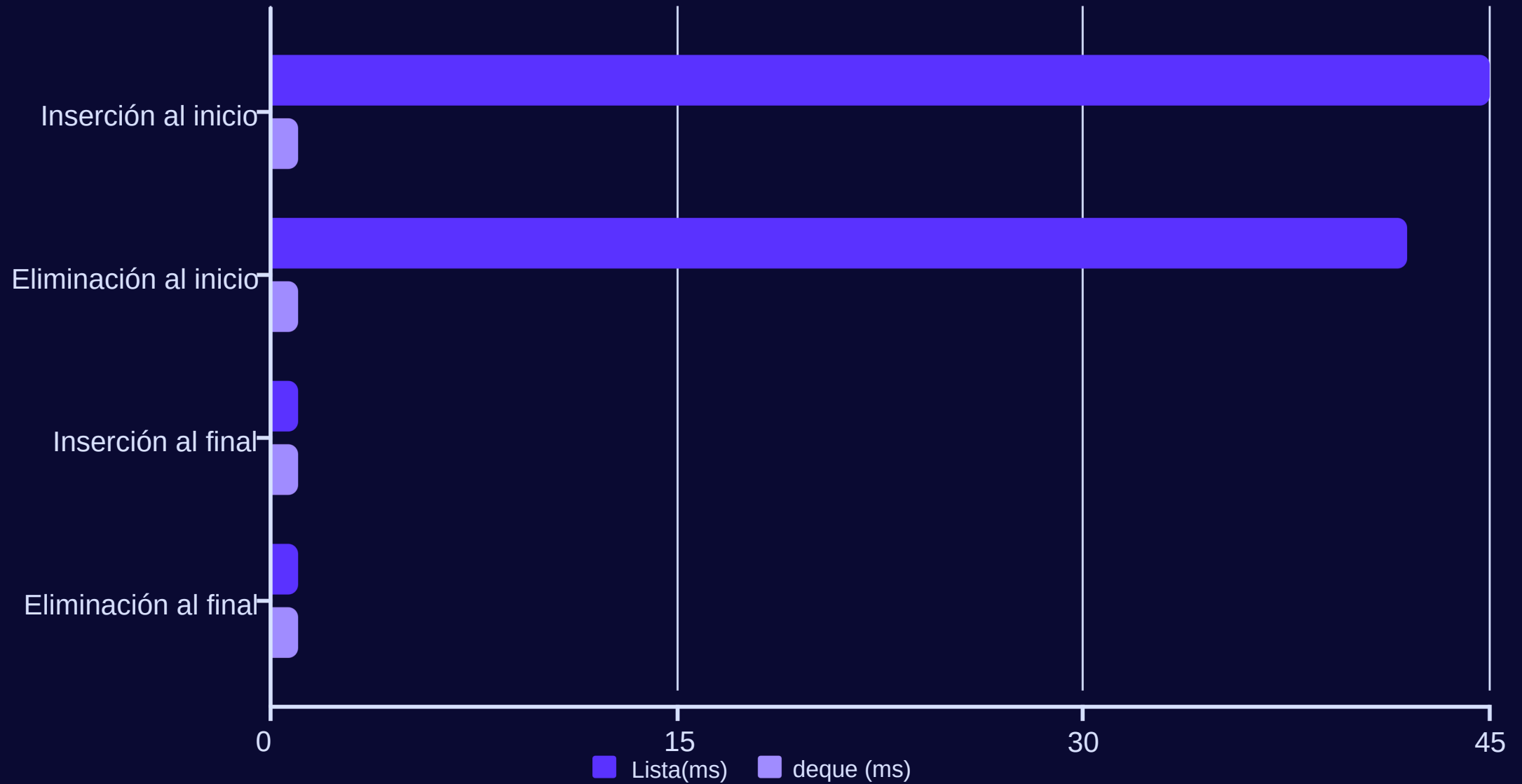
            # Determinar tiempo de ejecución
            tiempo_ejecucion = min(proceso_actual.tiempo_restante, quantum)
            proceso_actual.tiempo_restante -= tiempo_ejecucion
            tiempo_total += tiempo_ejecucion

            print(f"Ejecutando {proceso_actual.nombre} por {tiempo_ejecucion} unidades")

            # Si no terminó, volver a la cola
            if proceso_actual.tiempo_restante > 0:
                cola_listos.enqueue(proceso_actual)
            else:
                print(f"³ {proceso_actual.nombre} completado")

        print(f"Todos los procesos completados en {tiempo_total} unidades de tiempo")
```

Comparación: Listas vs deque



Los datos muestran claramente por qué deque es superior para operaciones de cola. La diferencia se vuelve crítica con estructuras grandes.



Colas de Prioridad

Las colas de prioridad permiten que elementos con mayor prioridad sean procesados antes, independientemente de su orden de llegada.

```
import heapq

class ColaPrioridad:
    def __init__(self):
        self.heap = []

    def enqueue(self, item, prioridad):
        """Menor número = mayor prioridad"""
        heapq.heappush(self.heap, (prioridad, item))

    def dequeue(self):
        if not self.heap:
            return None
        prioridad, item = heapq.heappop(self.heap)
        return item

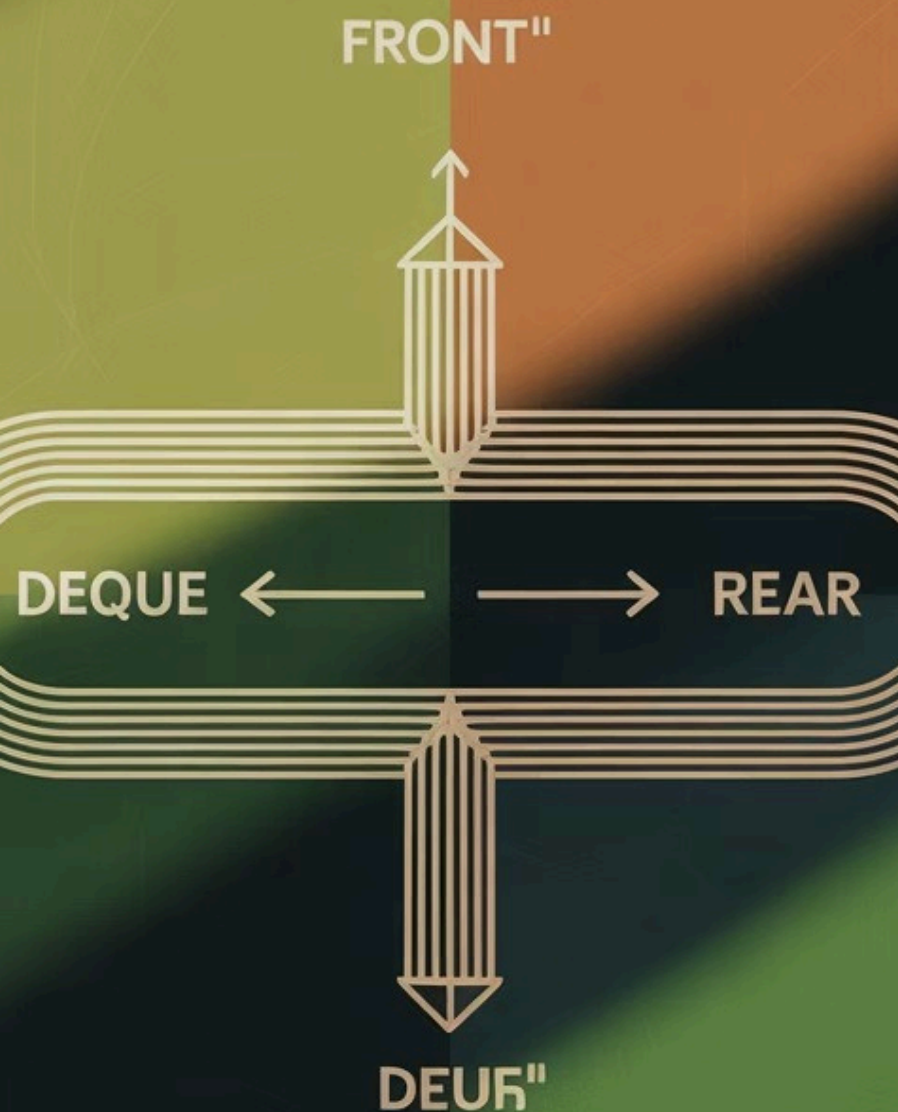
    def is_empty(self):
        return len(self.heap) == 0

# Ejemplo: Sistema de emergencias hospitalarias
hospital = ColaPrioridad()
hospital.enqueue("Dolor de cabeza", 5)
hospital.enqueue("Infarto", 1) # Mayor prioridad
hospital.enqueue("Fractura", 3)

print(hospital.dequeue()) # "Infarto" (prioridad 1)
```

Cola Doble (deque extendida)

La cola doble permite inserción y eliminación eficiente en ambos extremos, combinando las capacidades de pilas y colas.



```
from collections import deque

class ColaDoble:
    def __init__(self):
        self.items = deque()

    # Operaciones por el frente
    def add_front(self, item):
        self.items.appendleft(item)

    def remove_front(self):
        return self.items.popleft() if self.items else None

    # Operaciones por la parte trasera
    def add_rear(self, item):
        self.items.append(item)

    def remove_rear(self):
        return self.items.pop() if self.items else None

    def is_empty(self):
        return len(self.items) == 0

    # Ejemplo: Historial de navegación con deshacer/rehacer
    navegador = ColaDoble()
    navegador.add_rear("google.com")
    navegador.add_rear("github.com")
    navegador.add_front("stackoverflow.com") # Página anterior
```

Pila con Capacidad Máxima

```
class PilaCircular:
    def __init__(self, capacidad):
        self.capacidad = capacidad
        self.items = [None] * capacidad
        self.tope = -1
        self.tamaño = 0
        def push(self, item):

            if self.is_full():
                # Comportamiento circular: sobrescribir el más antiguo
                self.tope = (self.tope + 1) % self.capacidad
                self.items[self.tope] = item
            else:
                self.tope = (self.tope + 1) % self.capacidad
                self.items[self.tope] = item
                self.tamaño += 1

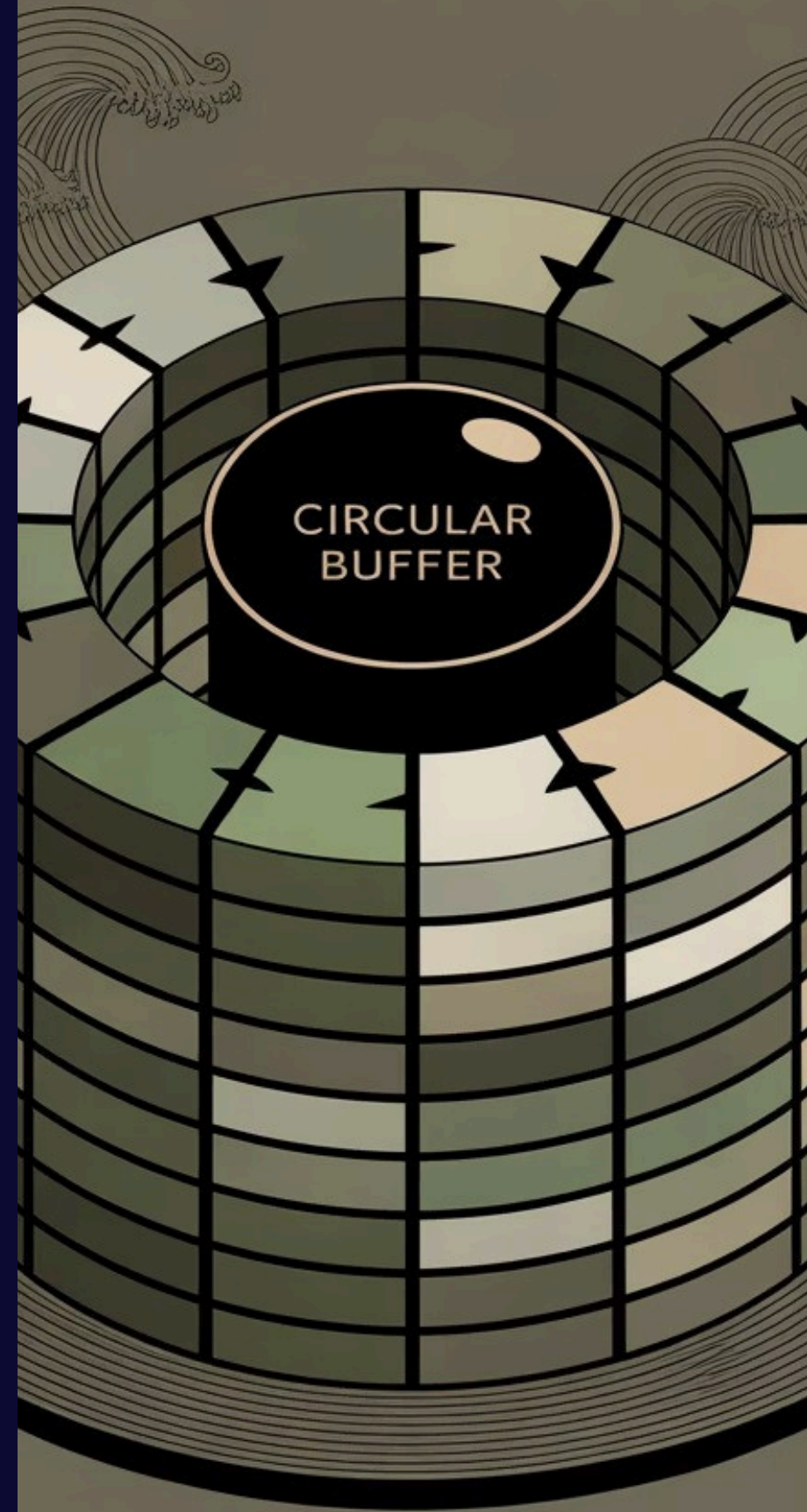
        def pop(self):
            if self.is_empty():
                return None

            item = self.items[self.tope]
            self.items[self.tope] = None
            self.tope = (self.tope - 1) % self.capacidad
            self.tamaño -= 1
            return item

        def is_full(self):
            return self.tamaño == self.capacidad

        def is_empty(self):
            return self.tamaño == 0
```

Útil en sistemas con memoria limitada donde necesitamos mantener solo los elementos más recientes.



Ejercicio Práctico 1: Invertir Cadena

Desafío: Implementa una función que invierta una cadena de texto utilizando una pila.

```
def invertir_cadena(texto):
    """Invierte una cadena usando una pila"""
    pila = Stack()
    # Apilar cada caracter
    for caracter in texto:

        pila.push(caracter)

    # Desapilar para formar la cadena invertida
    texto_invertido = ""
    while not pila.is_empty():
        texto_invertido += pila.pop()

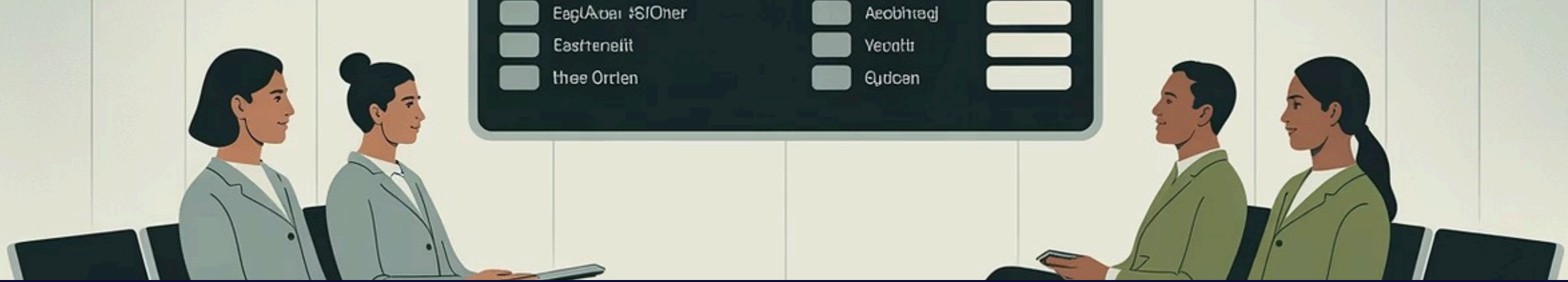
    return texto_invertido

# Pruebas
print(invertir_cadena("Python"))    # "nohtyP"
print(invertir_cadena("Hola Mundo")) # "odnuM aloH"
print(invertir_cadena("12345"))     # "54321"
```

¿Por qué funciona? La pila invierte naturalmente el orden debido al principio LIFO. El primer carácter en entrar es el último en salir.

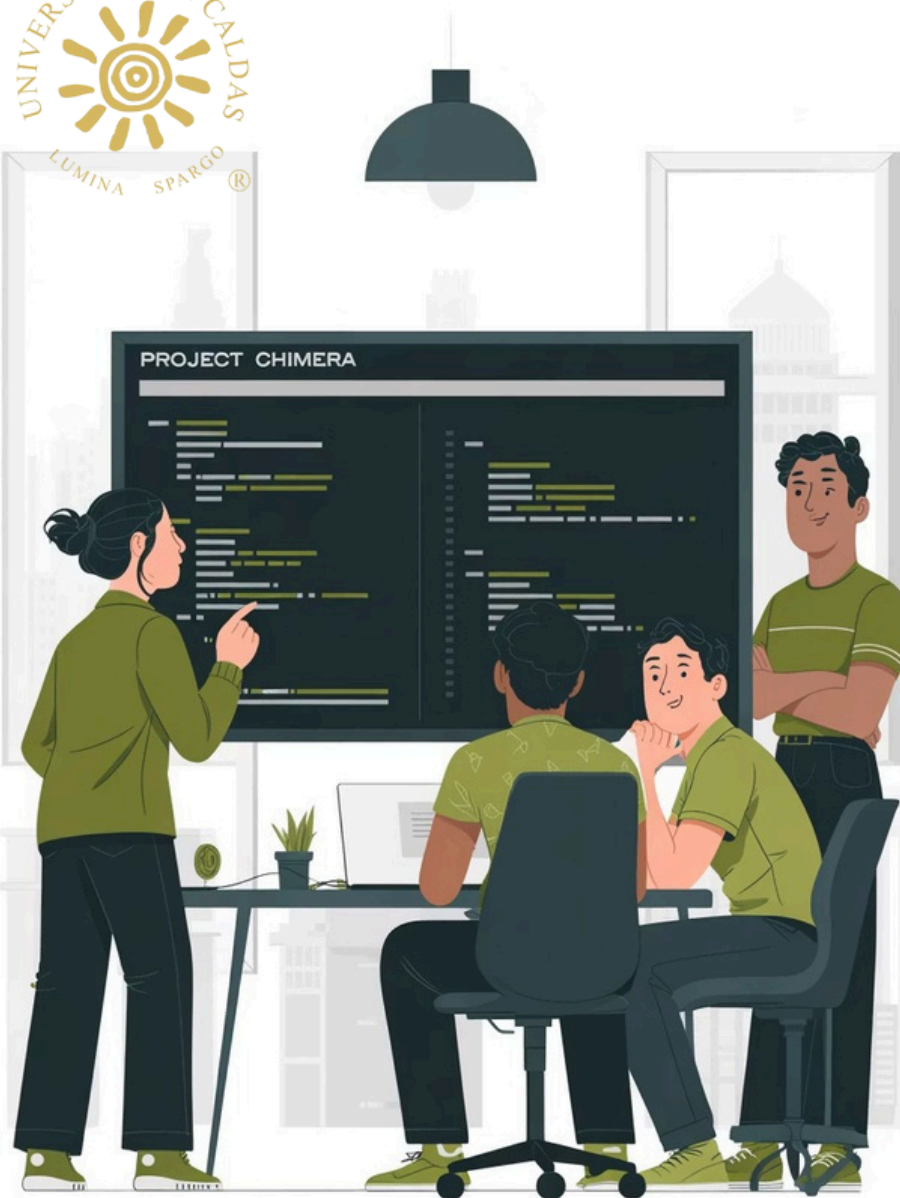
📌 **Extensión:** ¿Cómo modificarías esta función para invertir solo las palabras pero mantener su posición original?

String Reversal



Ejercicio Práctico 2: Sistema de Turnos Médicos

Escribir el código



Discusión Grupal: Análisis de Soluciones

1 Eficiencia de Algoritmos

¿Cómo cambia la complejidad temporal cuando usamos listas vs deque? Analicemos el impacto en aplicaciones reales con miles de operaciones.

2 Casos de Uso Alternativos

¿Qué otras aplicaciones pueden beneficiarse de pilas y colas? Pensemos en navegadores web, sistemas de chat, videojuegos.

3 Optimizaciones Posibles

¿Cómo podríamos mejorar nuestras implementaciones? Consideremos uso de memoria, manejo de errores, concurrencia.

4 Comparación con Otras Estructuras

¿Cuándo preferirías usar una lista enlazada, un array dinámico, o nuestras estructuras personalizadas?

Errores Comunes y Cómo Evitarlos



Pop en Estructura Vacía

Error: No verificar si la pila/cola está vacía antes de hacer pop/dequeue.

Solución: Siempre usar `is_empty()` o devolver None/lanzar excepción.

Uso Incorrecto de Índices

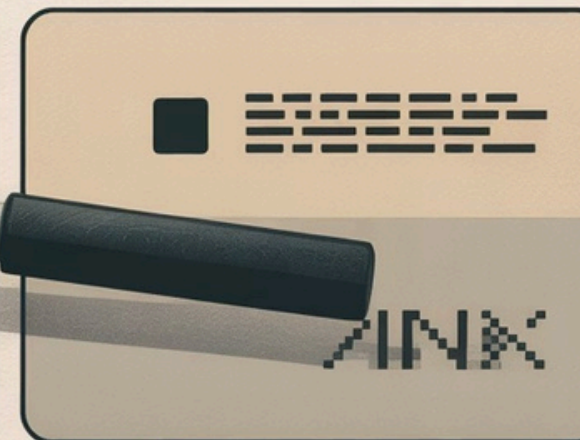
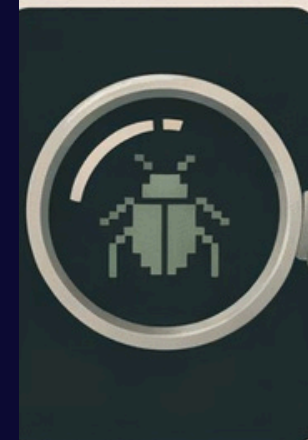
Error: Acceder directamente a índices en lugar de usar las operaciones apropiadas.

Solución: Respetar la interfaz de la estructura (solo push/pop para pilas).

Elección Incorrecta de Implementación

Error: Usar listas para colas en aplicaciones críticas de rendimiento.

Solución: Preferir `deque` para operaciones frecuentes en ambos extremos.





Buenas Prácticas de Implementación

01

Validación de Entrada

Siempre validar parámetros y manejar casos edge como estructuras vacías o elementos None.

02

Manejo de Excepciones

Decide si devolver None, lanzar excepciones personalizadas, o usar valores por defecto según el contexto.

03

Documentación Clara

Documentar la complejidad temporal, precondiciones y comportamiento esperado de cada método.

04

Preferencia por deque

Usa `collections.deque` cuando necesites operaciones eficientes en ambos extremos.

05

Pruebas Unitarias

Prueba casos límite: estructura vacía, un elemento, múltiples operaciones consecutivas.



Comparativa Final: Pilas vs Colas vs Deque

Característica	Pila	Cola	Deque
Principio	LIFO	FIFO	Ambos extremos
Inserción	Solo tope	Solo final	Ambos extremos
Eliminación	Solo tope	Solo inicio	Ambos extremos
Complejidad	$O(1)$ todas	$O(1)$ con deque	$O(1)$ todas
Uso típico	Recursión, deshacer	Planificación, BFS	Historial, buffers
Implementación	Lista Python	deque requerido	deque nativo



Cuándo Usar Cada Estructura



Usar Pilas Cuando...

- Necesites invertir orden
- Implementes recursión iterativa
- Manejes operaciones deshacer/rehacer
- Valides sintaxis (paréntesis, HTML)
- Implementes algoritmos de backtracking



Usar Colas Cuando...

- Necesites procesamiento FIFO
- Implementes BFS en grafos
- Manejes sistemas de turnos
- Gestionar buffers de datos
- Implementes planificadores de tareas



Usar Deque Cuando...

- Necesites flexibilidad en ambos extremos
- Implementes historial navegable
- Manejes sliding window algorithms
- Requieras eficiencia máxima
- Combines funcionalidad de pila y cola



Preguntas de Reflexión

Análisis de Complejidad

¿Por qué `pop(0)` en listas es $O(n)$ mientras que `popleft()` en deque es $O(1)$? ¿Qué implica esto para aplicaciones con millones de elementos?

Diseño de Sistemas

Si estuvieras diseñando un navegador web, ¿cómo implementarías el historial de páginas visitadas? ¿Usarías pila, cola, o deque? Justifica tu respuesta.

Optimización de Memoria

¿Cuándo sería apropiado implementar una pila o cola con capacidad limitada? ¿Qué ventajas y desventajas tendría este enfoque?

Mini-Reto Final: Implementaciones Avanzadas

Desafío 1: CircularQueue

Implementa una cola circular con capacidad fija. Cuando se llena, los nuevos elementos sobrescriben los más antiguos. Debe mantener eficiencia $O(1)$ en todas las operaciones.

 **Pista:** Usa un array de tamaño fijo con dos punteros (front y rear) que se mueven de forma circular usando el operador módulo (%).

Desafío 2: MinStack

Diseña una pila que además de las operaciones normales, permita obtener el elemento mínimo en tiempo $O(1)$. Debe funcionar incluso después de múltiples push y pop.

 **Pista:** Considera mantener una pila auxiliar que almacene los mínimos en cada nivel, o usa una estructura que almacene tanto el valor como el mínimo actual.

Estos desafíos te ayudarán a profundizar tu comprensión y aplicar creativamente los conceptos aprendidos. ¡Son problemas comunes en entrevistas técnicas!

¡Felicitaciones! Has Dominado Pilas y Colas

2

Estructuras
Fundamentales

Pilas y Colas dominadas
completamente

6

Operaciones Clave

Push, pop, enqueue, dequeue y
más

$O(1)$

Complejidad Óptima

Todas las operaciones en tiempo
constante

>

Aplicaciones Posibles

Desde algoritmos básicos hasta
sistemas complejos

"Las estructuras de datos no son solo herramientas técnicas, son formas de pensar y organizar la información. Dominar pilas y colas te abre las puertas a algoritmos más avanzados y soluciones más elegantes."

Próximos Pasos Recomendados

- Practica implementando los mini-retos propuestos
- Explora árboles y grafos usando estas estructuras como base
- Investiga algoritmos de ordenamiento que usan pilas
- Estudia patrones de diseño que aprovechan estas estructuras
- Aplica estos conceptos en proyectos reales

¡Continúa explorando el fascinante mundo de las estructuras de datos!